

Variance Reduction Techniques with Monte Carlo Simulation Method

An Application of the 4% Withdrawal Rule

Final Project Report

Evan Leonard

PART ONE

Description for a General Audience

The Big Question: Will Your Money Outlast You?

Imagine you have spent decades saving for retirement. You have built up perhaps \$500,000 or \$1,000,000 and now you are ready to stop working and start living off those savings. The next question to ask is simple but profound: will my money last as long as I do?

This project tackles that exact question using an approach called Monte Carlo simulation, a method that runs thousands of retirement scenarios to estimate how likely a retiree is to stay financially solvent throughout their lifetime.

The 4% Rule

In the 1990s, financial researcher William Bengen studied historical stock market data and concluded that a retiree could safely withdraw 4% of their starting portfolio each year (adjusting for inflation) with a high likelihood of never running out of money over a 30-year retirement. This became known as the 4% rule and remains one of the most widely referenced guidelines in retirement planning.

For example, if you retire with \$1,000,000, the 4% rule suggests withdrawing \$40,000 in the first year. The following year, you adjust that withdrawal upward by the inflation rate, and so on. The idea is that your remaining portfolio, invested in the stock market, will grow enough to sustain those withdrawals over time.

But here is an issue: the future is uncertain. Stock markets go up and down. You might live to 75 or you might live to 100. So instead of relying on historical averages alone, this project uses computer simulation to explore thousands of possible futures simultaneously.

How the Simulation Works (Baseline)

Think of each simulation as imagining the retirement of one person. For each imaginary retiree, the computer does the following:

- First, it randomly decides how long that person will live.
- Then, we take that future lifetime and find how many months there are in each retirement simulation.
- Then, it randomly simulates how the stock market performs each month.
- As it tracks the portfolio balance month by month, it is subtracting the monthly withdrawal and adjusting for market performance.
- If the balance hits zero before the person dies, they are considered insolvent and that is counted as a failure. If they pass away with money still in the account, that is a success and contributes towards finding an overall solvency rate.

By running this process 10,000 times, the simulation can produce a reliable solvency rate which is the percentage of simulated retirees whose money lasted their entire lifetime (remained solvent). This is the key number the project is trying to estimate as accurately as possible.

The Problem with Pure Randomness

A big issue with this approach is that, because everything is random, running the same simulation twice gives slightly different answers. In one run the imaginary retirees might get lucky. The market is doing well and lifetimes are shorter, which can produce a solvency rate of maybe 97%. In another run things go a bit worse by chance, and it comes out at 94%. Run it fifteen times and you get fifteen slightly different numbers.

For a financial professional advising a real retiree, that kind of variation is a problem. You want the simulation to give a stable, reliable answer that reflects the true underlying probability, not an answer that depends on whether the computer happened to draw good or bad random numbers on that run.

This project explored three different variance reduction techniques designed to make Monte Carlo simulation more stable and more accurate without simply running it more times.

The Four Techniques

1. The Original Simulation (Baseline)

The starting point is a plain Monte Carlo simulation where everything is purely random. This gave an average solvency rate of about 95.66% across fifteen runs, but the results varied by as much as 1.17 percentage points from run to run.

2. Antithetic Variate

This is the simplest of the three methods. Instead of running each retirement scenario once, it runs every scenario twice, once with the original random stock market drawings and once with their mirror images. Think of it like flipping a coin: if the first scenario gets heads, the second automatically gets tails. A great stock market run for the first scenario is automatically paired with a bad one for the second. Averaging the two outcomes removes a lot of the luck from the equation. This method reduced the run-to-run variation considerably.

3. Control Variate

This technique is certainly the most sophisticated of the three explored today. It works by identifying a related quantity that we know the true average of. In this case, a measure of how the stock market performed relative to what was expected. We use that expected performance to adjust our solvency estimate.

In plain terms: if the random stock market draws happened to be luckier than average in a given run, the raw solvency estimate will be slightly too high. If they were unluckier, the estimate will be slightly too low. The control variate method detects how lucky or unlucky the draws were and adjusts the estimate accordingly, essentially removing the luck from the equation.

This method is also expandable. We can introduce more 'control' variables to better adjust our final estimate. This was used in a second version of this method where we also accounted for whether the random lifetimes happened to run longer or shorter than average, correcting for luck in both the stock market randomness and the future lifetime randomness simultaneously. This did not change the average estimate but made it dramatically more stable, reducing run-to-run variation by a significant amount.

4. Stratified Sampling

Rather than drawing lifetimes completely at random, this method divides the full range of possible lifetimes into equal sections and forces the simulation to draw exactly one lifetime from each section. Imagine dividing a dartboard into 10,000 equal zones and requiring exactly one dart in each zone rather than throwing all the darts randomly. This guarantees that very short and very long lifetimes are always represented in every run, rather than leaving it to chance.

5. Combining Antithetic Variate with Stratified Sampling

You may have noticed that in antithetic variates, the method only affected the stock market randomness and with stratified sampling, it only affected the lifetime randomness. In noticing this, we can essentially combine the two techniques into an even better approach while still being relatively easy to integrate.

What We Found

All three variance reduction methods converged on a solvency rate of approximately 96.8%, compared to the baseline of 95.66%. Three completely independent methods arriving at the same number is strong evidence that 96.8% is close to the true underlying solvency probability for the parameters tested and that the plain simulation was slightly underestimating it.

The table below summarizes the results across fifteen runs of each method:

Method	Avg. Solvency	Std. Deviation	Range
Original Simulation	95.66%	0.2853	1.1700
Antithetic Variate	96.37%	0.1798	0.8650
Control Variate (1 control)	96.80%	0.1685	0.5934
Control Variate (2 controls)	96.79%	0.1094	0.4688
Stratified Sampling	96.80%	0.1419	0.4300
Antithetic + Stratified	96.83%	0.1315	0.4900

The key takeaway is that moving from the original simulation to any variance reduction method produces a meaningful improvement in both accuracy and stability. Adding more complexity beyond the first method gave only marginal further gains. For someone needing a quick, reliable estimate, the antithetic variate method offers the best return for its simplicity. If someone is looking into the most accurate estimate, the control variate is likely the best bet as it is scalable and highly interactive with inputs, though it is the most technically involved. The best of both worlds is likely the combination of antithetic variate and stratified sampling, having great returns with little variation and yet its implementation is not as complex as control variate can be.

Why Does This Matter?

The difference between a 95.66% and 96.80% solvency estimate might sound small, but in the context of retirement planning, especially with solvency estimates not as high as those reported here, it has serious implications. A financial planner using the underestimating plain simulation

might recommend unnecessarily conservative withdrawal rates, causing clients to spend less than they could afford to. A more accurate estimate gives better guidance.

More importantly, the reduction in simulation variation means that results from a single simulation run are more trustworthy. A planner can run the model once or just a couple times and have confidence the answer is stable, rather than needing to run it many times and average the results.

PART TWO

Technical Report for the Chief Actuary

1. Project Overview

This project implements and compares three Monte Carlo variance reduction techniques applied to the problem of estimating portfolio solvency probability under the 4% withdrawal rule. The simulation models two stochastic components jointly: future lifetime drawn from a Gompertz distribution, and portfolio evolution under lognormal stock price modeling. All simulations were implemented in Microsoft Excel VBA and run with $n = 10,000$ paths per trial, with results averaged across fifteen independent runs per method. All specification regarding variables in the Gompertz future lifetime model, lognormal stock modeling, and possible 4% rule portfolio adjustments were kept consistent throughout the entire process.

2. Model Specification

2.1 Mortality Model — Gompertz Distribution

Future lifetime T is modelled using the Gompertz distribution, characterised by the force of mortality:

$$\mu_x = B \times C^x$$

where B and C are the Gompertz parameters read from the spreadsheet. The survival function is:

$$S(t) = \exp(-B/\ln(C) \times C^x \times (C^t - 1))$$

Future lifetime is simulated via the inverse CDF (inverse transform method), where $u \sim \text{Uniform}(0,1)$:

$$T = (1/\ln C) \times \ln(1 - \ln(1-u) \times \ln(C) / (B \times C^{\text{Age}}))$$

The simulated lifetime is capped at $\max(0, 120 - \text{Age})$ years to prevent formula instability at extreme quantiles. The number of simulation months is computed as $m = \text{floor}(T \times 12)$.

2.2 Market Model — Geometric Brownian Motion

The stock price $S(t)$ follows Geometric Brownian Motion with drift $\alpha - \delta$ and volatility σ , where δ is the continuous dividend yield. The monthly lognormal increment is:

$$S(t+\Delta t) = S(t) \times \exp((\alpha - \delta - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t} \times Z)$$

where $Z \sim N(0,1)$ is drawn via the inverse CDF method applied to $u \sim \text{Uniform}(0,1)$, and $\Delta t = \text{TimeInc}$ represents the monthly time step (typically $1/12$). The standard normal variate is bounded away from 0 and 1 (clamped at 1×10^{-7} and $1 - 1 \times 10^{-7}$) to prevent numerical overflow in the NormInv function.

2.3 Portfolio Dynamics

For each simulated month k , the portfolio balance $B(k)$ evolves as:

$$B(k) = (S(k)/S(k-1)) \times B(k-1) - W(k)$$

where $W(k)$ is the monthly withdrawal. The initial withdrawal is $W(0) = \text{Bal0} \times \text{AnnualPercent} / 12$. At the end of each complete year (when $k/12$ is an integer), the withdrawal is inflated:

$$W(k) = W(k-1) \times (1 + \text{Inflation})$$

Ruin is declared if $B(k) < 0$ at any month $k \leq m$. The binary outcome indicator for path j is:

$$Y_j = 1 \text{ if } B(m) \geq 0, \quad Y_j = 0 \text{ if } B(k) < 0 \text{ for any } k \leq m$$

The plain Monte Carlo estimator of solvency probability P is:

$$\hat{P} = (1/n) \times \sum Y_j$$

3. Variance Reduction Methods

3.1 Antithetic Variate

For each path j , two correlated paths are simulated using the original standard normal draw Z and its negation $-Z$. Since Y is a monotone (or near-monotone) function of the stock draws, $Y(Z)$ and $Y(-Z)$ are negatively correlated, and their average has lower variance than either alone.

The antithetic estimator is:

$$\hat{P}_{AV} = (1/n) \times \sum (Y_j(Z) + Y_j(-Z)) / 2$$

Note that using $-Z$ is equivalent to applying $1-u$ to the uniform draw before NormInv , since $\text{NormInv}(1-u) = -\text{NormInv}(u)$. Both paths share the same Gompertz lifetime draw. The variance reduction is:

$$\text{Var}(\hat{P}_{AV}) = (1/2) \times (\text{Var}(Y) + \text{Cov}(Y(Z), Y(-Z)))$$

Variance reduction is achieved when $\text{Cov}(Y(Z), Y(-Z)) < 0$, which holds when Y is monotone in Z . The monthly inflation adjustment is applied identically to both paths.

3.2 Control Variate — Single Control (MkII)

The control variate method exploits a known analytical expectation to reduce estimator variance. The control variable is the standardised terminal stock price:

$$C_j = S(m_j) / E[S(m_j)] = S(m_j) / (S_0 \times \exp((\alpha - \delta) \times m_j \times \Delta t))$$

By construction, $E[C_j] = 1$ for all paths regardless of path length m_j , eliminating the variable-horizon confound present when using raw terminal stock price. The controlled estimator is:

$$\hat{P}_{CV} = \bar{Y} - \beta \times (\bar{C} - 1)$$

where $\bar{Y}$ and $\bar{C}$ are the sample means of the outcome and control respectively. The optimal beta is estimated from a pilot run of $n_{\text{pilot}} = \max(50, 0.3n)$ paths:

$$\beta = \text{Co}\hat{v}(Y, C) / \text{Va}\hat{r}(C) = [\hat{E}(YC) - \hat{E}(Y)\hat{E}(C)] / [\hat{E}(C^2) - \hat{E}(C)^2]$$

All expectations are computed using the standard computational formulae to avoid storing individual path values. Beta is clamped to $[-2, 2]$ to prevent overcorrection from noisy pilot estimates. The variance of the controlled estimator is:

$$\text{Var}(\hat{P}_{CV}) = \text{Var}(Y) \times (1 - \rho^2)$$

The correlation diagnostic $\rho = \text{Cor}(Y, C)$ is computed and output for each run. A rho of approximately 0.17 was observed, yielding a theoretical variance reduction of approximately 3%.

3.3 Control Variate — Two Controls (MkIII)

A second control variate is added using future lifetime T_j , with reference mean estimated from the pilot: $\hat{E}[T] = \bar{T}_{\text{pilot}}$. Since C_j and T_j are correlated (longer lifetimes produce more months of stock growth), the optimal betas must be solved jointly via the 2×2 linear system:

$$\begin{bmatrix} \text{Var}(C) & \text{Cov}(C,T) \\ \text{Cov}(C,T) & \text{Var}(T) \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} \text{Cov}(Y,C) \\ \text{Cov}(Y,T) \end{bmatrix}$$

Solved via Cramer's rule using the determinant $\text{Det} = \text{Var}(C) \times \text{Var}(T) - \text{Cov}(C,T)^2$. If $|\text{Det}| < 10^{-10}$ both betas are set to zero to prevent numerical instability. The two-control estimator is:

$$\hat{P}_{CV2} = \bar{Y} - \beta_1(\bar{C} - 1) - \beta_2(\bar{T} - \hat{E}[T])$$

Adding the lifetime control did not shift the point estimate (both single and double control converged to about 96.80%) but reduced the standard deviation from 0.1685 to 0.1094, confirming that FutLife captures an independent source of simulation noise not addressed by the stock control alone.

3.4 Stratified Sampling

The uniform random draws used for the Gompertz lifetime inversion are replaced with stratified draws. The $[0,1]$ interval is partitioned into n equal strata of width $1/n$. One uniform draw is taken from the j -th stratum:

$$u_j = (j - 1 + U_j) / n, \quad U_j \sim \text{Uniform}(0,1)$$

The n stratified values are written to a worksheet column and subjected to a Fisher-Yates shuffle prior to the main simulation loop to randomise path ordering without destroying stratification. The shuffle requires $O(n)$ swaps and uses the standard algorithm:

$$\text{For } j = n \text{ to } 2: \text{ swap } u_j \text{ with } u_k, k \sim \text{Uniform}\{1, \dots, j\}$$

For smooth functions of the lifetime draw, stratified sampling reduces variance at the faster rate $O(1/n^2)$ versus the standard $O(1/n)$ rate of plain Monte Carlo. In practice, because the monthly stock draws ($O(m)$ per path) dominate total variance and are not stratified, the observable improvement is more modest than theory predicts for a single stratified dimension.

3.5 Combined Antithetic Variate and Stratified Sampling

The combined method applies stratified sampling to the lifetime draw and antithetic variate to the stock path draws simultaneously. The $n/2$ stratified lifetime values are shared between antithetic pairs. Each pair runs both the original and mirror-image stock paths for the same stratified lifetime draw. Both paths are updated within a single monthly loop to avoid temporary storage overhead, which was identified as a significant performance bottleneck in earlier implementations that wrote monthly draws to worksheet cells.

The estimator is:

$$\hat{P}_{\text{Comb}} = (2/n) \times \sum_j (Y_j(Z, T_{\text{strat}}) + Y_j(-Z, T_{\text{strat}})) / 2$$

4. Results and Discussion

4.1 Summary of Results

Method	Avg. Solvency	Std. Dev.	Range	Var. Red. vs Base
Original (Plain MC)	95.66%	0.2853	1.1700	—
Antithetic Variate	96.37%	0.1798	0.8650	60.3%
CV (1 control)	96.80%	0.1685	0.5934	65.1%
CV (2 controls)	96.79%	0.1094	0.4688	85.3%
Stratified Sampling	96.75%	0.1920	0.7300	54.7%
Antithetic + Stratified	96.83%	0.1315	0.4900	78.8%

Variance reduction percentages are computed as $1 - (\text{SD}^2_{\text{method}} / \text{SD}^2_{\text{base}})$. All results are averaged across fifteen independent runs with $n = 10,000$ paths per run.

4.2 Convergence of Point Estimates

The three methodologically independent approaches, single control variate, two-control variate, and stratified sampling, all converged to a solvency rate of approximately 96.80%, compared to the plain Monte Carlo estimate of 95.66%. This convergence across independent estimators constitutes strong evidence that 96.80% is close to the true theoretical solvency probability under the given parameter set.

The 1.14 percentage point gap between the plain Monte Carlo estimator and the corrected estimators is attributable to Jensen's inequality bias. The solvency probability is a concave nonlinear function of the random inputs in the neighbourhood of the true probability. For finite n , the sample mean of a nonlinear function of random variables is systematically biased toward the concave side. Variance reduction methods, by smoothing out the random variation, effectively correct for this finite-sample bias and produce estimates closer to the true expectation.

4.3 Accuracy vs Precision — The Two-Control Result

A key finding is the distinction between accuracy (point estimate correction) and precision (variance reduction). The first control variate corrected the point estimate from 95.66% to 96.80%, a relatively large accuracy improvement. Adding the lifetime control variate left the point estimate unchanged at 96.79% but reduced the variance by 57.8%. This is theoretically expected: across many runs the correction terms for both controls average to zero, so the mean estimate is

unaffected. Within any single run, however, the lifetime correction removes the noise contribution from lifetime draw variability, tightening the distribution of estimates around the true value.

4.4 Practical Ranking by Efficiency

Considering both variance reduction and implementation complexity:

- The two-control variate achieved the highest variance reduction (85.4%) but required a pilot run, 2×2 matrix inversion, and separate output cells for four diagnostic quantities.
- Stratified sampling achieved 75.2% reduction with minimal code complexity — a single pre-simulation loop plus Fisher-Yates shuffle.
- Antithetic variate achieved 60.2% reduction with a single additional line of code per monthly iteration, making it the most efficient method per unit of implementation effort.
- The combined antithetic and stratified method achieved 78.9% reduction, slightly below the two-control variate, with moderate implementation complexity.

For a practitioner seeking reliable variance reduction with minimal implementation overhead, antithetic variate is recommended as the first-line method, with antithetic + stratified sampling as a far more accurate but slightly more intensive supplement. The two-control variate is recommended when maximum precision is required and the pilot run overhead is acceptable.

4.5 Interpretation of Rho and Beta

The correlation diagnostic rho for the standardised stock price control was observed to be slightly negative (approximately -0.05 to -0.15 across runs). This indicates that higher-than-expected terminal stock price is weakly negatively correlated with survival in this parameter regime. At a solvency rate of approximately 96.80%, almost all paths survive, and the few failures tend to be driven by extreme adverse sequences in the early retirement period rather than systematically poor terminal stock performance. In this regime, any stock-based control variate will exhibit weak correlation with the binary outcome, limiting variance reduction regardless of sign.

Beta values for the stock control were small in magnitude (approximately 0.03 – 0.05), consistent with the scale ratio between the bounded binary outcome and the standardised stock price. Beta values for the lifetime control were larger in magnitude and negative, reflecting that longer lifetimes are associated with higher ruin probability (more months of withdrawal exposure) which is the economically expected relationship.

5. Spreadsheet Description

The project is implemented in Microsoft Excel with one sheet per method:

- Var1 — Base Monte Carlo simulation (VarOne macro)
- AntitheticVariate — Antithetic variate simulation (AntitheticVariate macro)
- ControlVariate — Control variate simulations, both single and two-control versions (ControlVariateMkII and ControlVariateMkIII macros)
- StratifiedSampling — Stratified sampling simulation (StratifiedSamplingMkI macro)
- Antithetic+Stratified — Combined method (AntiStratMkI macro)

APPENDIX — VBA Macro Code

A.1 Base Monte Carlo — VarOne

```
Sub VarOne()

    Sheets("Var1").Select
    Range("A7:B10008").ClearContents

    Let A = Excel.Cells(2, 4)
    Let B = Excel.Cells(3, 4)
    Let C = Excel.Cells(4, 4)
    Let n = Excel.Cells(1, 6)
    Age = Excel.Cells(2, 6)

    Alpha = Excel.Cells(1, 2)
    Sigma = Excel.Cells(2, 2)
    Delta = Excel.Cells(3, 2)
    SO = Excel.Cells(4, 2) 'Stock Price at time 0
    TimeInc = Excel.Cells(5, 2)

    Bal0 = Excel.Cells(3, 6)
    AnnualPercent = Excel.Cells(4, 6)
    Inflation = Excel.Cells(6, 6)
    Count = 0

    Application.Calculation = xlCalculationManual 'Speeds up macro

    'Stochastic on Stochastic simulation to determine the probability of a positive
    balance at the end of life.

    For j = 1 To n

        'This generates a randomly chosen value from future lifetime and PV of WL
        insurance benefit
        Randomize Timer
        v = Rnd
        FutLife = (1 / Log(C)) * Log(1 - (Log(1 - v) * Log(C)) / (B * (C ^ Age)))
        'Create formula using the inverse CDF for the Gompertz Distribution

        'Initial values for each trial
        MonthlyPay = Bal0 * AnnualPercent / 12
        m = Int(FutLife * 12) 'Months to simulate market
        StkPrice = SO
        Bal = Bal0

        'm trials
        For k = 1 To m

            StkPricePrior = StkPrice 'Beginning of month stock price
            Randomize Timer
            v = Rnd
            y = WorksheetFunction.NormInv(v, 0, 1)

            StkPrice = StkPricePrior * Exp((Alpha - Delta - 0.5 * Sigma * Sigma) *
            TimeInc + Sigma * Sqr(TimeInc) * y) 'Use Lognormal Models to produce stock price
            for the next month

            Bal = StkPrice / StkPricePrior * Bal 'Producing the end of month
            balance in the ETF
```

```

        Bal = Bal - MonthlyPay           'Balance after the withdrawal
        If Bal < 0 Then GoTo 50           'Test to see if the balance is
negative. If negative it records bal, does not increment count, next trial
        If Int(k / 12) = k / 12 Then
            MonthlyPay = MonthlyPay * (1 + Inflation)
        End If

    Next k
    Excel.Cells(8 + j, 2) = FutLife
50    Excel.Cells(8 + j, 1) = Bal
        If Bal > 0 Then Count = Count + 1 'Count is the number of trials that end
with a positive balance at death

Next j

Excel.Cells(7, 2) = (Count / n) * 100

Application.Calculation = xlCalculationAutomatic
Worksheets("Var1").Calculate
'

End Sub

```

A.2 Antithetic Variate — AntitheticVariate

```

Sub AntitheticVariate()

    Sheets("AntitheticVariate").Select
    Range("A7:B10008").ClearContents

    Let A = Excel.Cells(2, 4)
    Let B = Excel.Cells(3, 4)
    Let C = Excel.Cells(4, 4)
    Let n = Excel.Cells(1, 6)
    Age = Excel.Cells(2, 6)
    Alpha = Excel.Cells(1, 2)
    Sigma = Excel.Cells(2, 2)
    Delta = Excel.Cells(3, 2)
    SO = Excel.Cells(4, 2)
    TimeInc = Excel.Cells(5, 2)
    Bal0 = Excel.Cells(3, 6)
    AnnualPercent = Excel.Cells(4, 6)
    Inflation = Excel.Cells(6, 6)

    Count = 0
    Application.Calculation = xlCalculationManual

    For j = 1 To n

        ' Draw a random future lifetime from the Gompertz distribution
        Randomize Timer
        v = Rnd
        FutLife = (1 / Log(C)) * Log(1 - (Log(1 - v) * Log(C)) / (B * (C ^ Age)))

        MonthlyPay = Bal0 * AnnualPercent / 12
        m = Int(FutLife * 12)
        StkPrice1 = SO
        StkPrice2 = SO
        Bal1 = Bal0
        Bal2 = Bal0
        Ruined1 = False
    
```

```

Ruined2 = False

' Simulate the market month by month for this person's lifetime
For k = 1 To m
    StkPricePrior1 = StkPrice1
    StkPricePrior2 = StkPrice2
    Randomize Timer
    v = Rnd
    y = WorksheetFunction.NormInv(v, 0, 1)
    Y_antithetic = -y

    ' adding in -Z stock path
    StkPrice1 = StkPricePrior1 * Exp((Alpha - Delta - 0.5 * Sigma * Sigma)
* TimeInc + Sigma * Sqr(TimeInc) * y)
    StkPrice2 = StkPricePrior2 * Exp((Alpha - Delta - 0.5 * Sigma * Sigma)
* TimeInc + Sigma * Sqr(TimeInc) * Y_antithetic)

    ' stop updating once ruined (<0)
    If Not Ruined1 Then
        Bal1 = StkPrice1 / StkPricePrior1 * Bal1 - MonthlyPay
        If Bal1 < 0 Then Ruined1 = True
    End If

    If Not Ruined2 Then
        Bal2 = StkPrice2 / StkPricePrior2 * Bal2 - MonthlyPay
        If Bal2 < 0 Then Ruined2 = True
    End If

    If Int(k / 12) = k / 12 Then
        MonthlyPay = MonthlyPay * (1 + Inflation)
    End If

Next k

'average the outcome (1/0), not the raw dollar balance
Outcome1 = 0
Outcome2 = 0
If Not Ruined1 Then Outcome1 = 1
If Not Ruined2 Then Outcome2 = 1
AvgOutcome = (Outcome1 + Outcome2) / 2

Excel.Cells(8 + j, 2) = FutLife
If Not Ruined1 Then
    Excel.Cells(8 + j, 1) = Bal1
Else
    Excel.Cells(8 + j, 1) = -1
End If

Count = Count + AvgOutcome

Next j

Excel.Cells(7, 2) = (Count / n) * 100
Application.Calculation = xlCalculationAutomatic
Worksheets("AntitheticVariate").Calculate

End Sub

```

A.3 Control Variate Single Control — ControlVariateMkII

```
Sub ControlVariateMkII()
```

```

Sheets("ControlVariate").Select
Range("A9:B10008").ClearContents
Range("H12:H13").ClearContents

Let A = Excel.Cells(2, 4)
Let B = Excel.Cells(3, 4)
Let C = Excel.Cells(4, 4)
Let n = Excel.Cells(1, 6)
Age = Excel.Cells(2, 6)
Alpha = Excel.Cells(1, 2)
Sigma = Excel.Cells(2, 2)
Delta = Excel.Cells(3, 2)
SO = Excel.Cells(4, 2)
TimeInc = Excel.Cells(5, 2)
Bal0 = Excel.Cells(3, 6)
AnnualPercent = Excel.Cells(4, 6)
Inflation = Excel.Cells(6, 6)

Application.Calculation = xlCalculationManual

Randomize

' PHASE 1 - PILOT RUN
n_pilot = Application.WorksheetFunction.Max(50, Int(0.3 * n))

' Control: Standardized stock price using
SumOut = 0
SumSm = 0
SumSm2 = 0
SumOutSm = 0

For jp = 1 To n_pilot

    v = Rnd
    FutLife = (1 / Log(C)) * Log(1 - (Log(1 - v) * Log(C)) / (B * (C ^ Age)))
    If FutLife < 0 Then FutLife = 0
    If FutLife > 120 - Age Then FutLife = 120 - Age

    m = Int(FutLife * 12)

```

```

    StkPrice = SO
    Bal = Bal0
    MonthlyPay = Bal0 * AnnualPercent / 12
    Outcome = 0
    bRuined = False

    For k = 1 To m
        v = Rnd
        y = WorksheetFunction.NormInv(v, 0, 1)
        StkPricePrior = StkPrice
        StkPrice = StkPricePrior * Exp((Alpha - Delta - 0.5 * Sigma * Sigma) *
TimeInc + Sigma * Sqr(TimeInc) * y)
        Bal = (StkPrice / StkPricePrior) * Bal - MonthlyPay
        If Bal < 0 Then
            bRuined = True
            Exit For
        End If
        If Int(k / 12) = k / 12 Then MonthlyPay = MonthlyPay * (1 + Inflation)
    Next k

    If Not bRuined Then Outcome = 1

PilotDone:

    ESm = SO * Exp((Alpha - Delta) * m * TimeInc) 'ESm = expected stock price at
end of lifetime.
    If ESm > 0 Then
        StdSm = StkPrice / ESm 'StdSm = ratio observed stock/Expected stock.
    Else
        StdSm = 1
    End If

    SumOut = SumOut + Outcome ' Finding total survivors (remained solvent)
    SumSm = SumSm + StdSm ' Will be used to find average standardized stock
    SumSm2 = SumSm2 + StdSm * StdSm ' Will be used to compute variance without
having to store every value
    SumOutSm = SumOutSm + Outcome * StdSm ' Will be used to compute covariance
between survival outcome and standardized stock price

Next jp

```

```

' Computing Beta
MeanOut_p = SumOut / n_pilot ' Raw survival prob
MeanSm_p = SumSm / n_pilot ' Average standardized stock price
CovOutSm = (SumOutSm / n_pilot) - MeanOut_p * MeanSm_p ' Covariance = E[XY] -
E[X]E[Y]
VarSm = (SumSm2 / n_pilot) - MeanSm_p * MeanSm_p ' Variance of standardized stock
= E[X^2] - E[X]^2

If VarSm <> 0 Then
    beta = CovOutSm / VarSm
Else
    beta = 0
End If

' Clamp beta to prevent overcorrection
If beta > 2 Then beta = 2
If beta < -2 Then beta = -2

MeanESm_ref = 1

' Correlation Diagnostic
VarOut_p = MeanOut_p * (1 - MeanOut_p)
If VarOut_p > 0 And VarSm > 0 Then
    rho = CovOutSm / Sqr(VarOut_p * VarSm) ' Rho tells correlation, how tightly
these variables move together
Else
    rho = 0
End If

' PHASE 2 - MAIN RUN
SumOutMain = 0 ' Same as SumOut but for main run
SumSmMain = 0 ' Same as SumSm but for main run

For j = 1 To n

    v = Rnd
    FutLife = (1 / Log(C)) * Log(1 - (Log(1 - v) * Log(C)) / (B * (C ^ Age)))
    If FutLife < 0 Then FutLife = 0
    If FutLife > 120 - Age Then FutLife = 120 - Age

    m = Int(FutLife * 12)

```

```

StkPrice = SO
Bal = Bal0
MonthlyPay = Bal0 * AnnualPercent / 12
Outcome = 0
bRuined = False

For k = 1 To m
    v = Rnd
    y = WorksheetFunction.NormInv(v, 0, 1)
    StkPricePrior = StkPrice
    StkPrice = StkPricePrior * Exp((Alpha - Delta - 0.5 * Sigma * Sigma) *
TimeInc + Sigma * Sqr(TimeInc) * y)
    Bal = (StkPrice / StkPricePrior) * Bal - MonthlyPay
    If Bal < 0 Then
        bRuined = True
        Exit For
    End If
    If Int(k / 12) = k / 12 Then MonthlyPay = MonthlyPay * (1 + Inflation)
Next k

If Not bRuined Then Outcome = 1

MainDone:
Excel.Cells(8 + j, 1) = Bal
Excel.Cells(8 + j, 2) = FutLife

' Standardized stock price for main run
ESm = SO * Exp((Alpha - Delta) * m * TimeInc)
If ESm > 0 Then
    StdSm = StkPrice / ESm
Else
    StdSm = 1
End If

SumOutMain = SumOutMain + Outcome
SumSmMain = SumSmMain + StdSm

Next j

```

```

' Final Estimates
MeanOut_main = SumOutMain / n
MeanSm_main = SumSmMain / n

P_CV = MeanOut_main - beta * (MeanSm_main - MeanESm_ref) ' (MeanSm_main -
MeanESm_ref) is essentially how lucky/unlucky stock draws were

' Did the stock draws run luckier than average? ? term is positive ? raw outcome
count is inflated ? subtract to correct

' Did the stock draws run unluckier than average? ? term is negative ? raw outcome
count is deflated ? add back to correct

' Did the stock draws run exactly as expected? ? term is zero ? no correction
needed ? P_CV equals plain Monte Carlo

' Our control variates find a beta that signifies how much we adjust our main
output P_CV by how lucky or unlucky our stock draws were

' essentially removing the luck from the equation (for stock draws not lifetime
draws)

If P_CV < 0 Then P_CV = 0
If P_CV > 1 Then P_CV = 1

Excel.Cells(9, 8) = P_CV * 100
Excel.Cells(10, 8) = beta
Excel.Cells(11, 8) = rho

Application.Calculation = xlCalculationAutomatic

End Sub

```

A.4 Control Variate Two Controls — ControlVariateMkIII

```

Sub ControlVariateMkIII()

    Sheets("ControlVariate").Select

    Range("A9:B10008").ClearContents

    Let A = Excel.Cells(2, 4)

    Let B = Excel.Cells(3, 4)

    Let C = Excel.Cells(4, 4)

```

```

Let n = Excel.Cells(1, 6)

Age = Excel.Cells(2, 6)

Alpha = Excel.Cells(1, 2)

Sigma = Excel.Cells(2, 2)

Delta = Excel.Cells(3, 2)

SO = Excel.Cells(4, 2)

TimeInc = Excel.Cells(5, 2)

Bal0 = Excel.Cells(3, 6)

AnnualPercent = Excel.Cells(4, 6)

Inflation = Excel.Cells(6, 6)


Application.Calculation = xlCalculationManual


Randomize


' PHASE 1 - PILOT RUN


n_pilot = Application.WorksheetFunction.Max(50, Int(0.3 * n))


SumOut = 0 ' Total survival outcomes in pilot
SumSm = 0 ' Total standardized stock prices
SumSm2 = 0 ' Used for variance of stock control
SumOutSm = 0 ' Used for calculations later
SumFL = 0 ' Total of all future lifetime values
SumFL2 = 0 ' Used for variance of lifetime control
SumOutFL = 0 ' Used for calculations later
SumSmFL = 0 ' Used for correlation between controls


For jp = 1 To n_pilot

```

```

v = Rnd

FutLife = (1 / Log(C)) * Log(1 - (Log(1 - v) * Log(C)) / (B * (C ^ Age)))

If FutLife < 0 Then FutLife = 0

If FutLife > 120 - Age Then FutLife = 120 - Age

m = Int(FutLife * 12)

StkPrice = SO

Bal = Bal0

MonthlyPay = Bal0 * AnnualPercent / 12

Outcome = 0

bRuined = False

For k = 1 To m

    v = Rnd

    If v < 0.0000001 Then v = 0.0000001

    If v > 0.9999999 Then v = 0.9999999

    y = WorksheetFunction.NormInv(v, 0, 1)

    StkPricePrior = StkPrice

    StkPrice = StkPricePrior * Exp((Alpha - Delta - 0.5 * Sigma * Sigma) *
TimeInc + Sigma * Sqr(TimeInc) * y)

    Bal = (StkPrice / StkPricePrior) * Bal - MonthlyPay

    If Bal < 0 Then

        bRuined = True

        Exit For

    End If

    If Int(k / 12) = k / 12 Then MonthlyPay = MonthlyPay * (1 + Inflation)

Next k

If Not bRuined Then Outcome = 1

```

```

ESm = SO * Exp((Alpha - Delta) * m * TimeInc)

If ESm > 0 Then

    StdSm = StkPrice / ESm

Else

    StdSm = 1

End If

' Accumulate for both control variates

SumOut = SumOut + Outcome

SumSm = SumSm + StdSm

SumSm2 = SumSm2 + StdSm * StdSm

SumOutSm = SumOutSm + Outcome * StdSm

SumFL = SumFL + FutLife

SumFL2 = SumFL2 + FutLife * FutLife

SumOutFL = SumOutFL + Outcome * FutLife

SumSmFL = SumSmFL + StdSm * FutLife

Next jp

' Compute two betas using 2x2 matrix solution

MeanOut_p = SumOut / n_pilot ' pilot survival rate

MeanSm_p = SumSm / n_pilot ' avgerage stock "luck"

MeanFL_p = SumFL / n_pilot ' average lifetime in pilot, this is the second control

CovOutSm = (SumOutSm / n_pilot) - MeanOut_p * MeanSm_p ' How stock luck and
survival move together

CovOutFL = (SumOutFL / n_pilot) - MeanOut_p * MeanFL_p ' How lifetime luck and
survival move together

VarSm = (SumSm2 / n_pilot) - MeanSm_p * MeanSm_p ' Variance of stock control

VarFL = (SumFL2 / n_pilot) - MeanFL_p * MeanFL_p ' Variance of lifetime control

CovSmFL = (SumSmFL / n_pilot) - MeanSm_p * MeanFL_p ' Correlation between
variables

```

```
' Solve 2x2 system: [VarSm, CovSmFL; CovSmFL, VarFL] * [b1; b2] = [CovOutSm;  
CovOutFL]
```

```
Det = VarSm * VarFL - CovSmFL * CovSmFL
```

```
If Abs(Det) > 0.0000000001 Then
```

```
    beta1 = (CovOutSm * VarFL - CovOutFL * CovSmFL) / Det
```

```
    beta2 = (CovOutFL * VarSm - CovOutSm * CovSmFL) / Det
```

```
Else
```

```
    beta1 = 0
```

```
    beta2 = 0
```

```
End If
```

```
' Clamp both betas
```

```
If beta1 > 2 Then beta1 = 2
```

```
If beta1 < -2 Then beta1 = -2
```

```
If beta2 > 2 Then beta2 = 2
```

```
If beta2 < -2 Then beta2 = -2
```

```
' Reference means
```

```
MeanESm_ref = 1          ' E[StdSm] = 1 by construction
```

```
MeanFL_ref = MeanFL_p    ' Use pilot mean as reference for FutLife
```

```
' --- Correlation diagnostics ---
```

```
VarOut_p = MeanOut_p * (1 - MeanOut_p)
```

```
If VarOut_p > 0 And VarSm > 0 Then
```

```
    rho_Sm = CovOutSm / Sqr(VarOut_p * VarSm)
```

```
Else
```

```
    rho_Sm = 0
```

```
End If
```

```
If VarOut_p > 0 And VarFL > 0 Then
```

```
    rho_FL = CovOutFL / Sqr(VarOut_p * VarFL)
```

```

Else

    rho_FL = 0

End If

' PHASE 2 - MAIN RUN

SumOutMain = 0

SumSmMain = 0

SumFLMain = 0

For j = 1 To n

    v = Rnd

    FutLife = (1 / Log(C)) * Log(1 - (Log(1 - v) * Log(C)) / (B * (C ^ Age)))

    If FutLife < 0 Then FutLife = 0

    If FutLife > 120 - Age Then FutLife = 120 - Age

    m = Int(FutLife * 12)

    StkPrice = S0

    Bal = Bal0

    MonthlyPay = Bal0 * AnnualPercent / 12

    Outcome = 0

    bRuined = False

    For k = 1 To m

        v = Rnd

        If v < 0.0000001 Then v = 0.0000001

        If v > 0.9999999 Then v = 0.9999999

        y = WorksheetFunction.NormInv(v, 0, 1)

        StkPricePrior = StkPrice

```

```

        StkPrice = StkPricePrior * Exp((Alpha - Delta - 0.5 * Sigma * Sigma) *
TimeInc + Sigma * Sqr(TimeInc) * y)

        Bal = (StkPrice / StkPricePrior) * Bal - MonthlyPay

        If Bal < 0 Then

            bRuined = True

            Exit For

        End If

        If Int(k / 12) = k / 12 Then MonthlyPay = MonthlyPay * (1 + Inflation)
Next k

If Not bRuined Then Outcome = 1

Excel.Cells(8 + j, 1) = Bal

Excel.Cells(8 + j, 2) = FutLife

' Standardized stock price for main run
ESm = SO * Exp((Alpha - Delta) * m * TimeInc)

If ESm > 0 Then

    StdSm = StkPrice / ESm

Else

    StdSm = 1

End If

SumOutMain = SumOutMain + Outcome ' Total survival outcomes in main simulation

SumSmMain = SumSmMain + StdSm ' Total standardized stock prices in main
simulation

SumFLMain = SumFLMain + FutLife ' Total future lifetimes in main simulation

Next j

```

```

' Final estimates

MeanOut_main = SumOutMain / n

MeanSm_main = SumSmMain / n

MeanFL_main = SumFLMain / n

P_CV = MeanOut_main - beta1 * (MeanSm_main - MeanESm_ref) - beta2 * (MeanFL_main -
MeanFL_ref) ' Subtracting "luck" from both stock priceing and future lifetime

If P_CV < 0 Then P_CV = 0

If P_CV > 1 Then P_CV = 1

' Output

Excel.Cells(9, 8) = P_CV * 100           ' CV probability %

Excel.Cells(10, 8) = beta1               ' Beta1 (stock)

Excel.Cells(12, 8) = beta2               ' Beta2 (lifetime)

Excel.Cells(11, 8) = rho_Sm              ' Rho for stock CV

Excel.Cells(13, 8) = rho_FL              ' Rho for lifetime CV

Application.Calculation = xlCalculationAutomatic

End Sub

```

A.5 Stratified Sampling — StratifiedSamplingMkI

```

Sub StratifiedSamplingMkI()

' STRATIFIED SAMPLING SUMMARY

' In a standard simulation, all random draws come from Rnd which is purely

' random. By chance, some regions of the probability space get oversampled

' and others get undersampled, causing the estimate to vary between runs.

' Stratified sampling fixes this by dividing the [0,1] probability space

' into n equal-width buckets (strata) and drawing exactly one random value

' from each bucket. For example with n=4 buckets:

'   Bucket 1: draw from [0.00, 0.25)

```

```
' Bucket 2: draw from [0.25, 0.50)
' Bucket 3: draw from [0.50, 0.75)
' Bucket 4: draw from [0.75, 1.00)
' This guarantees the full range of lifetimes is always represented in every
' run. Short lifetimes, medium lifetimes and long lifetimes all get sampled
' in proportion every single time, rather than by luck.
' The buckets are then shuffled randomly so paths are not processed in
' sorted lifetime order, which could introduce bias in the balance
' calculations.
```

```
Sheets("StratifiedSampling").Select
Range("A7:B10008").ClearContents
```

```
Let A = Excel.Cells(2, 4)
Let B = Excel.Cells(3, 4)
Let C = Excel.Cells(4, 4)
Let n = Excel.Cells(1, 6)
Age = Excel.Cells(2, 6)
Alpha = Excel.Cells(1, 2)
Sigma = Excel.Cells(2, 2)
Delta = Excel.Cells(3, 2)
SO = Excel.Cells(4, 2)
TimeInc = Excel.Cells(5, 2)
Bal0 = Excel.Cells(3, 6)
AnnualPercent = Excel.Cells(4, 6)
Inflation = Excel.Cells(6, 6)
Count = 0

Application.Calculation = xlCalculationManual
```

```
' STEP 1 - BUILD THE STRATIFIED SAMPLE VALUES INTO COLUMN C
```

```
' We write one stratified value per row into column C (rows 9 to 8+n).
```

```
' Each row j gets one draw from its own exclusive bucket of the [0,1]
```

```
' interval. The formula:
```

```
'      (j - 1 + Rnd) / n
```

```
' places a random point inside bucket j. For example with n=1000:
```

```
'      j=1  draws from [0.000, 0.001)
```

```
'      j=2  draws from [0.001, 0.002)
```

```
'      j=500 draws from [0.499, 0.500)  etc.
```

```
' This ensures every slice of the lifetime distribution is sampled.
```

```
Randomize Timer ' Seed the random number generator once before building strata
```

```
For j = 1 To n
```

```
    ' Place one random point inside the j-th equal-width bucket
```

```
    ' and write it to column C for use in the main loop
```

```
    Excel.Cells(8 + j, 3) = (j - 1 + Rnd) / n
```

```
Next j
```

```
' STEP 2 - SHUFFLE THE STRATIFIED VALUES (Fisher-Yates Shuffle)
```

```
' Without shuffling the paths would be processed in order of increasing
```

```
' lifetime, which could create subtle patterns in the balance outputs.
```

```
' The Fisher-Yates shuffle randomly reorders the column C values so each
```

```
' path gets a stratified lifetime value but in a completely random order.
```

```

For j = n To 2 Step -1
    ' Pick a random index from 1 to j
    SwapIdx = Int(Rnd * j) + 1

    ' Swap the value at row j with the value at the randomly chosen row
    Temp = Excel.Cells(8 + j, 3)
    Excel.Cells(8 + j, 3) = Excel.Cells(8 + SwapIdx, 3)
    Excel.Cells(8 + SwapIdx, 3) = Temp
Next j

' STEP 3 - MAIN SIM

' Each path j reads its pre-assigned stratified value from column C
' instead of calling Rnd for the lifetime draw. The stock price path
' inside the monthly loop still uses pure Rnd since stratifying thousands
' of monthly draws per path is not practical.

For j = 1 To n

    ' LIFETIME DRAW (Stratified)

    ' Read the pre-assigned stratified value from column C for this path
    ' instead of using Rnd, guaranteeing full coverage of all lifetimes
    v = Excel.Cells(8 + j, 3)

    FutLife = (1 / Log(C)) * Log(1 - (Log(1 - v) * Log(C)) / (B * (C ^ Age)))

    ' Cap lifetime at sensible boundaries to avoid formula errors
    If FutLife < 0 Then FutLife = 0

```

```

If FutLife > 120 - Age Then FutLife = 120 - Age

MonthlyPay = Bal0 * AnnualPercent / 12

m = Int(FutLife * 12)

StkPrice = S0

Bal = Bal0

' MONTHLY SIMULATION LOOP

' Simulate the stock price and portfolio balance month by month
' for the full duration of this path's lifetime
For k = 1 To m

    StkPricePrior = StkPrice

    'STOCK DRAW (not stratified)

    ' Each monthly stock move is drawn purely randomly since there
    ' are too many monthly draws to stratify individually

    v = Rnd

    ' Clamp v away from 0 and 1 to prevent NormInv from crashing

    If v < 0.0000001 Then v = 0.0000001

    If v > 0.9999999 Then v = 0.9999999

    y = WorksheetFunction.NormInv(v, 0, 1)

    StkPrice = StkPricePrior * Exp((Alpha - Delta - 0.5 * Sigma * Sigma) *
TimeInc + Sigma * Sqr(TimeInc) * y)

    Bal = (StkPrice / StkPricePrior) * Bal

    Bal = Bal - MonthlyPay

    If Bal < 0 Then GoTo 50

```

```

        If Int(k / 12) = k / 12 Then

            MonthlyPay = MonthlyPay * (1 + Inflation)

        End If

    Next k

50    Excel.Cells(8 + j, 1) = Bal

        Excel.Cells(8 + j, 2) = FutLife

        If Bal > 0 Then Count = Count + 1

    Next j

Excel.Cells(7, 2) = (Count / n) * 100 ' Survival probability as a percentage

Application.Calculation = xlCalculationAutomatic ' Re-enable automatic
calculation

Worksheets("StratifiedSampling").Calculate

End Sub

```

A.6 Combined Antithetic and Stratified — AntiStratMkI

```

Sub AntiStratMkI()

    'Combining antithetic variates, which targets stock path draws,
    'with stratified sampling, which targets future lifetime draws,
    'we can reduce variance on both random draws and hopefully
    'further reduce variance overall than either could achieve separately

    Sheets("Antithetic+Stratified").Select
    Range("A7:C10008").ClearContents

    ' Read all parameters from the spreadsheet
    Let A = Excel.Cells(2, 4)
    Let B = Excel.Cells(3, 4)
    Let C = Excel.Cells(4, 4)
    Let n = Excel.Cells(1, 6)
    Age = Excel.Cells(2, 6)
    Alpha = Excel.Cells(1, 2)
    Sigma = Excel.Cells(2, 2)
    Delta = Excel.Cells(3, 2)
    SO = Excel.Cells(4, 2)

```

```

TimeInc = Excel.Cells(5, 2)
Bal0 = Excel.Cells(3, 6)
AnnualPercent = Excel.Cells(4, 6)
Inflation = Excel.Cells(6, 6)
Count = 0

' Force n to be even so every path has an antithetic partner
If n Mod 2 <> 0 Then n = n + 1

Application.Calculation = xlCalculationManual

' STEP 1 - BUILD STRATIFIED VALUES INTO COLUMN C
' Only n/2 stratified values needed since each value is shared
' between two paths - the original and its antithetic partner

Randomize Timer

For j = 1 To n / 2
    ' Draw one random point from the j-th bucket of width 2/n
    Excel.Cells(8 + j, 3) = (j - 1 + Rnd) / (n / 2)
Next j

' STEP 2 - SHUFFLE THE STRATIFIED VALUES (Fisher-Yates Shuffle)
' Randomly reorder so pairs are not processed in sorted lifetime order

For j = (n / 2) To 2 Step -1
    SwapIdx = Int(Rnd * j) + 1
    Temp = Excel.Cells(8 + j, 3)
    Excel.Cells(8 + j, 3) = Excel.Cells(8 + SwapIdx, 3)
    Excel.Cells(8 + SwapIdx, 3) = Temp
Next j

' STEP 3 - Main Sim

For j = 1 To n Step 2

    v_life = Excel.Cells(8 + Int((j + 1) / 2), 3)

    FutLife = (1 / Log(C)) * Log(1 - (Log(1 - v_life) * Log(C)) / (B * (C ^
Age)))
    If FutLife < 0 Then FutLife = 0
    If FutLife > 120 - Age Then FutLife = 120 - Age
    m = Int(FutLife * 12)

    ' INITIALISE BOTH PATHS BEFORE THE MONTHLY LOOP

    ' Path A - original stock draws
    StkPrice_A = S0
    Bal_A = Bal0
    MonthlyPay_A = Bal0 * AnnualPercent / 12
    bRuined_A = False

    ' Path B - antithetic stock draws, identical starting values
    StkPrice_B = S0
    Bal_B = Bal0
    MonthlyPay_B = Bal0 * AnnualPercent / 12
    bRuined_B = False

    ' MONTHLY LOOP - Both paths updated simultaneously

    ' v is drawn once per month. Path A uses v, Path B uses 1-v.

```

```

' Both balances are updated in the same iteration with no storage needed.
' A path that has already been ruined is skipped to save computation.

For k = 1 To m

    ' Draw one random value for this month – used by both paths
    v = Rnd
    If v < 0.0000001 Then v = 0.0000001
    If v > 0.9999999 Then v = 0.9999999

    ' PATH A update (if not already ruined)
    If Not bRuined_A Then
        ' Original draw for Path A
        y_A = WorksheetFunction.NormInv(v, 0, 1)
        StkPricePrior_A = StkPrice_A
        StkPrice_A = StkPricePrior_A * Exp((Alpha - Delta - 0.5 * Sigma *
Sigma) * TimeInc + Sigma * Sqr(TimeInc) * y_A)
        Bal_A = (StkPrice_A / StkPricePrior_A) * Bal_A - MonthlyPay_A
        If Bal_A < 0 Then bRuined_A = True
    End If

    ' PATH B update (if not already ruined)
    If Not bRuined_B Then
        ' Antithetic draw for Path B – mirror image of Path A's draw
        y_B = WorksheetFunction.NormInv(1 - v, 0, 1)
        StkPricePrior_B = StkPrice_B
        StkPrice_B = StkPricePrior_B * Exp((Alpha - Delta - 0.5 * Sigma *
Sigma) * TimeInc + Sigma * Sqr(TimeInc) * y_B)
        Bal_B = (StkPrice_B / StkPricePrior_B) * Bal_B - MonthlyPay_B
        If Bal_B < 0 Then bRuined_B = True
    End If

    ' Exit the month loop early if both paths are already ruined
    If bRuined_A And bRuined_B Then Exit For

    ' Increase withdrawal by inflation at each year end for active paths
    If Int(k / 12) = k / 12 Then
        If Not bRuined_A Then MonthlyPay_A = MonthlyPay_A * (1 + Inflation)
        If Not bRuined_B Then MonthlyPay_B = MonthlyPay_B * (1 + Inflation)
    End If

Next k

' Determine survival outcome for each path
If Not bRuined_A Then Outcome_A = 1 Else Outcome_A = 0
If Not bRuined_B Then Outcome_B = 1 Else Outcome_B = 0

' Record results for both paths
Excel.Cells(8 + j, 1) = Bal_A
Excel.Cells(8 + j, 2) = FutLife
If j + 1 <= n Then
    Excel.Cells(8 + j + 1, 1) = Bal_B
    Excel.Cells(8 + j + 1, 2) = FutLife
End If

' AVERAGE THE PAIR
Count = Count + (Outcome_A + Outcome_B) / 2

Next j

' STEP 4 – Output

```

```
Excel.Cells(7, 2) = (Count / (n / 2)) * 100
```

```
Application.Calculation = xlCalculationAutomatic  
Worksheets("Var1").Calculate
```

```
End Sub
```